Pandas.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

RAM Disk

Share

[2] ✓ 0s

```
#Pandas - Python for Data Analysis

import numpy as np

import pandas as pd

# Data -
# variable - a = 10 (single data type)
# Sequential data - List , Tuple, Set, Dict
# Array - numpy
# Data - always in 2D- Rows & Columns
```

[3] ✓ 0s

```
dict = {"Name": ["Amit", "Rahul", "Priya"],
        "Age":[25,30,28],
        "Salary": [30000, 45000, 50000]}
```

[4] ✓ 0s

```
dict
```

```
{'Name': ['Amit', 'Rahul', 'Priya'],
 'Age': [25, 30, 28],
 'Salary': [30000, 45000, 50000]}
```

[ ]

```
# pd - Pndas - 1 - DataFrame , 2 -Series
```

[5] ✓ 0s

```
df = pd.DataFrame(dict)
```

[6] ✓ 0s

```
df
```

| | Name | Age | Salary |
|---|---|---|---|
| 0 | Amit | 25 | 30000 |
| 1 | Rahul | 30 | 45000 |
| 2 | Priya | 28 | 50000 |

Next steps: Generate code with df | New interactive sheet

[7] ✓ 0s

```
# Attributed
```

[8] ✓ 0s

```
df.shape
```

```
(3, 3)
```

[9] ✓ 0s

```
df.size
```

```
9
```

[10] ✓ 0s

```
df.columns    # very important
```

```
Index(['Name', 'Age', 'Salary'], dtype='object')
```

[11] ✓ 0s

```
df.dtypes
```

| | 0 |
|---|---|
| Name | object |
| Age | int64 |
| Salary | int64 |

dtype: object

[12] ✓ 0s

```
df.index
```

```
RangeIndex(start=0, stop=3, step=1)
```

[14] ✓ 0s

```
df.ndim  # n no of dimention
```

```
2
```

[15] ✓ 0s

```
# Function
```

[16] ✓ 0s

```
df.head()   # by default print the 5 lines from the top
#by default print 5 lines from table  - limit 5, LIMIT 10
# filter top N, powerbi FIlter - top 10 , 7 ,
```

| | Name | Age | Salary |
|---|---|---|---|
| 0 | Amit | 25 | 30000 |
| 1 | Rahul | 30 | 45000 |
| 2 | Priya | 28 | 50000 |

[17] ✓ 0s

```
df.head(1)
```

| | Name | Age | Salary |
|---|---|---|---|

| | Name | Age | Salary |
|---|---|---|---|
| 0 | Amit | 25 | 30000 |

Next steps: Generate code with df | New interactive sheet

[18] ✓ 0s

```
df.tail()
# print 5 last lines by default
```

| | Name | Age | Salary |
|---|---|---|---|
| 0 | Amit | 25 | 30000 |
| 1 | Rahul | 30 | 45000 |
| 2 | Priya | 28 | 50000 |

[19] ✓ 0s

```
df.tail(2)
```

| | Name | Age | Salary |
|---|---|---|---|
| 1 | Rahul | 30 | 45000 |
| 2 | Priya | 28 | 50000 |

[23]

```
df.sample()  # by default print any one line randomly-
```

| | Name | Age | Salary |
|---|---|---|---|
| 0 | Amit | 25 | 30000 |

[24] ✓ 0s

```
df.sample(2)   # randomly print any two lines
```

| | Name | Age | Salary |
|---|---|---|---|
| 2 | Priya | 28 | 50000 |
| 1 | Rahul | 30 | 45000 |

[25] ✓ 0s

```
df.info()    # very important
# information print
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3 entries, 0 to 2
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Name    3 non-null      object
 1   Age     3 non-null      int64
 2   Salary  3 non-null      int64
dtypes: int64(2), object(1)
memory usage: 204.0+ bytes
```

[26] ✓ 0s

```
df.describe()
#summery of all numerical columns / statistical summery
```

| | Age | Salary |
|---|---|---|
| count | 3.000000 | 3.000000 |
| mean | 27.666667 | 41666.666667 |
| std | 2.516611 | 10408.329997 |
| min | 25.000000 | 30000.000000 |
| 25% | 26.500000 | 37500.000000 |
| 50% | 28.000000 | 45000.000000 |
| 75% | 29.000000 | 47500.000000 |
| max | 30.000000 | 50000.000000 |

[27] ✓ 0s

```
# for categarical also for statistical description
```

[28] ✓ 0s

```
df.describe(include="all")
```

| | Name | Age | Salary |
|---|---|---|---|
| count | 3 | 3.000000 | 3.000000 |
| unique | 3 | NaN | NaN |
| top | Amit | NaN | NaN |
| freq | 1 | NaN | NaN |
| mean | NaN | 27.666667 | 41666.666667 |
| std | NaN | 2.516611 | 10408.329997 |
| min | NaN | 25.000000 | 30000.000000 |
| 25% | NaN | 26.500000 | 37500.000000 |
| 50% | NaN | 28.000000 | 45000.000000 |
| 75% | NaN | 29.000000 | 47500.000000 |
| max | NaN | 30.000000 | 50000.000000 |

[29] ✓ 0s

```
# indexing
# list - dict- dict["keys"]
```

[30] ✓ 0s

```
df["Name"]
```

| | Name |
|---|---|
| 0 | Amit |
| 1 | Rahul |
| 2 | Priya |

**dtype:** object

[31] ✓ 0s

```
df["Age"]
```

| | Age |
|---|---|
| 0 | 25 |
| 1 | 30 |
| 2 | 28 |

**dtype:** int64

[32] ✓ 0s

```
# two columns
# pass the list inside the indexing
```

[34] ✓ 0s

```
df[["Name", "Age"]]
```

| | Name | Age |
|---|---|---|
| 0 | Amit | 25 |
| 1 | Rahul | 30 |
| 2 | Priya | 28 |

[35] ✓ 0s

```
# for row indexing
```

[36] ✓ 0s

```
# loc - iloc
# location - index location
# Index name  like row 1 - location
# index no. -like 0 ,  - index location
```

[37] ✓ 0s

```
df.loc[1]
```

| | 1 |
|---|---|
| Name | Rahul |
| Age | 30 |
| Salary | 45000 |

**dtype:** object

[42] ✓ 0s

```
df1 = pd.DataFrame(dict , index= ["row1", "row2", "row3"])
```

[43] ✓ 0s

```
df1
```

| | Name | Age | Salary |
|---|---|---|---|
| row1 | Amit | 25 | 30000 |
| row2 | Rahul | 30 | 45000 |
| row3 | Priya | 28 | 50000 |

Next steps: Generate code with df1 | New interactive sheet

[44] ✓ 0s

```
df1.iloc[1]
```

| | row2 |
|---|---|
| Name | Rahul |
| Age | 30 |
| Salary | 45000 |

**dtype:** object

[45] ✓ 0s

```
df1.loc["row2"]
```

| | row2 |
|---|---|
| Name | Rahul |
| Age | 30 |
| Salary | 45000 |

**dtype:** object

[46] ✓ 0s

```
# loc = label base index
# iloc - index no. location
```

[47] ✓ 0s

```
# filtering data
```

[48] ✓ 0s

```
df
```

| | Name | Age | Salary |
|---|---|---|---|

| | Name | Age | Salary |
|---|---|---|---|
| 0 | Amit | 25 | 30000 |
| 1 | Rahul | 30 | 45000 |
| 2 | Priya | 28 | 50000 |

Next steps: Generate code with df | New interactive sheet

[49]
```
df[df["Salary"] > 40000]
```

| | Name | Age | Salary |
|---|---|---|---|
| 1 | Rahul | 30 | 45000 |
| 2 | Priya | 28 | 50000 |

[50]
```
df[(df["Salary"] > 40000) & ( df["Age"] >= 30)]

# AND condition & - both condition true
```

| | Name | Age | Salary |
|---|---|---|---|
| 1 | Rahul | 30 | 45000 |

[51]
```
df[(df["Salary"] > 40000) | ( df["Age"] >= 30)]

# OR - | - any one condition
```

| | Name | Age | Salary |
|---|---|---|---|
| 1 | Rahul | 30 | 45000 |
| 2 | Priya | 28 | 50000 |

[ ]
```
# and - &
# or - |
# not - ~
```

[52]
```
# isin()   # mysql - in - extact match
```

[53]
```
df[df["Name"].isin(["Amit", "Priya"])]
```

| | Name | Age | Salary |
|---|---|---|---|
| 0 | Amit | 25 | 30000 |
| 2 | Priya | 28 | 50000 |

[54]
```
df[df["Salary"].between(30000, 40000)]
```

| | Name | Age | Salary |
|---|---|---|---|
| 0 | Amit | 25 | 30000 |

[55]
```
df
```

| | Name | Age | Salary |
|---|---|---|---|
| 0 | Amit | 25 | 30000 |
| 1 | Rahul | 30 | 45000 |
| 2 | Priya | 28 | 50000 |

Next steps: Generate code with df | New interactive sheet

[56]
```
# sorting
```

[58]
```
df3 = df.sort_values("Age")
```

[59]
```
df3
```

| | Name | Age | Salary |
|---|---|---|---|
| 0 | Amit | 25 | 30000 |
| 2 | Priya | 28 | 50000 |
| 1 | Rahul | 30 | 45000 |

Next steps: Generate code with df3 | New interactive sheet

[60]
```
df3.sort_index()
```

| | Name | Age | Salary |
|---|---|---|---|
| 0 | Amit | 25 | 30000 |
| 1 | Rahul | 30 | 45000 |
| 2 | Priya | 28 | 50000 |

[61]
```
df4 = df3.reset_index()  # very important -
```

[62]
```
df4
```

| | index | Name | Age | Salary |
|---|---|---|---|---|
| 0 | 0 | Amit | 25 | 30000 |
| 1 | 2 | Priya | 28 | 50000 |
| 2 | 1 | Rahul | 30 | 45000 |

Next steps: Generate code with df4 | New interactive sheet

[64]
```
df3.reset_index(drop=True)   # delete the old index
```

| | Name | Age | Salary |
|---|---|---|---|
| 0 | Amit | 25 | 30000 |
| 1 | Priya | 28 | 50000 |
| 2 | Rahul | 30 | 45000 |

[65]
```
df["Pramotion"] =  df["Salary"] * 0.25
```

[66]
```
df
```

| | Name | Age | Salary | Pramotion |
|---|---|---|---|---|
| 0 | Amit | 25 | 30000 | 7500.0 |
| 1 | Rahul | 30 | 45000 | 11250.0 |
| 2 | Priya | 28 | 50000 | 12500.0 |

Next steps: Generate code with df | New interactive sheet

[67]
```
df["Final_Salary"] = df["Salary"] + df["Pramotion"]
```

[68]
```
df
```

| | Name | Age | Salary | Pramotion | Final_Salary |
|---|---|---|---|---|---|
| 0 | Amit | 25 | 30000 | 7500.0 | 37500.0 |
| 1 | Rahul | 30 | 45000 | 11250.0 | 56250.0 |
| 2 | Priya | 28 | 50000 | 12500.0 | 62500.0 |

Next steps: Generate code with df | New interactive sheet

[69]
```
df.rename(columns= {"Name" : "FirstName" } )
```

| | FirstName | Age | Salary | Pramotion | Final_Salary |
|---|---|---|---|---|---|
| 0 | Amit | 25 | 30000 | 7500.0 | 37500.0 |
| 1 | Rahul | 30 | 45000 | 11250.0 | 56250.0 |
| 2 | Priya | 28 | 50000 | 12500.0 | 62500.0 |

[70]
```
df.drop("Final_Salary", axis=1)  # axis 1 use for the column
```

| | Name | Age | Salary | Pramotion |
|---|---|---|---|---|
| 0 | Amit | 25 | 30000 | 7500.0 |
| 1 | Rahul | 30 | 45000 | 11250.0 |
| 2 | Priya | 28 | 50000 | 12500.0 |

[71]
```
df
```

| | Name | Age | Salary | Pramotion | Final_Salary |
|---|---|---|---|---|---|
| 0 | Amit | 25 | 30000 | 7500.0 | 37500.0 |
| 1 | Rahul | 30 | 45000 | 11250.0 | 56250.0 |
| 2 | Priya | 28 | 50000 | 12500.0 | 62500.0 |

Next steps: Generate code with df | New interactive sheet

[72]
```
df.drop("Final_Salary", axis=1 , inplace=True)
#inplace = True - for permenant drop/delete
```

[73]
```
df
```

| | Name | Age | Salary | Pramotion |
|---|---|---|---|---|
| 0 | Amit | 25 | 30000 | 7500.0 |
| 1 | Rahul | 30 | 45000 | 11250.0 |
| 2 | Priya | 28 | 50000 | 12500.0 |

Next steps: Generate code with df | New interactive sheet

[75]
```
df.drop(index= 0, axis = 0)   # axis 0  for the row
```

| | Name | Age | Salary | Pramotion |
|---|---|---|---|---|
| 1 | Rahul | 30 | 45000 | 11250.0 |

| | | | | |
|---|---|---|---|---|
| 2 | Priya | 28 | 50000 | 12500.0 |

[76] ✓ 0s

```
df
```

| | Name | Age | Salary | Pramotion |
|---|---|---|---|---|
| 0 | Amit | 25 | 30000 | 7500.0 |
| 1 | Rahul | 30 | 45000 | 11250.0 |
| 2 | Priya | 28 | 50000 | 12500.0 |

Next steps: Generate code with df | New interactive sheet

[77] ✓ 0s

```
df
```

| | Name | Age | Salary | Pramotion |
|---|---|---|---|---|
| 0 | Amit | 25 | 30000 | 7500.0 |
| 1 | Rahul | 30 | 45000 | 11250.0 |
| 2 | Priya | 28 | 50000 | 12500.0 |

Next steps: Generate code with df | New interactive sheet

[79] ✓ 0s

```
df =  pd.DataFrame({
    "Name":["Amit", "Rahul", "Priya", "Neha"],
    "Age" :[25 , None, 28, 35],
    "Salary" :[30000,45000, None, 60000]
})
```

[80] ✓ 0s

```
df
```

| | Name | Age | Salary |
|---|---|---|---|
| 0 | Amit | 25.0 | 30000.0 |
| 1 | Rahul | NaN | 45000.0 |
| 2 | Priya | 28.0 | NaN |
| 3 | Neha | 35.0 | 60000.0 |

Next steps: Generate code with df | New interactive sheet

[81] ✓ 0s

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4 entries, 0 to 3
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Name    4 non-null      object
 1   Age     3 non-null      float64
 2   Salary  3 non-null      float64
dtypes: float64(2), object(1)
memory usage: 228.0+ bytes
```

[82] ✓ 0s

```
df.isnull()
```

| | Name | Age | Salary |
|---|---|---|---|
| 0 | False | False | False |
| 1 | False | True | False |
| 2 | False | False | True |
| 3 | False | False | False |

[83] ✓ 0s

```
df.isnull().sum()   # very important - null values - missing values
```

| | 0 |
|---|---|
| Name | 0 |
| Age | 1 |
| Salary | 1 |

dtype: int64

[84] ✓ 0s

```
df.isna()
```

| | Name | Age | Salary |
|---|---|---|---|
| 0 | False | False | False |
| 1 | False | True | False |
| 2 | False | False | True |
| 3 | False | False | False |

[85] ✓ 0s

```
df.isna().sum()
```

| | 0 |
|---|---|
| Name | 0 |
| Age | 1 |

| | |
|---|---|
| Salary | 1 |

dtype: int64

[86]
```
df.notna()
```

| | Name | Age | Salary |
|---|---|---|---|
| 0 | True | True | True |
| 1 | True | False | True |
| 2 | True | True | False |
| 3 | True | True | True |

[87]
```
df.notna().sum()
```

| | 0 |
|---|---|
| Name | 4 |
| Age | 3 |
| Salary | 3 |

dtype: int64

[ ]
```
# drop specific nan column
```

[89]
```
df.dropna(subset=["Salary"] )   # very important - delete the null columns
```

| | Name | Age | Salary |
|---|---|---|---|
| 0 | Amit | 25.0 | 30000.0 |
| 1 | Rahul | NaN | 45000.0 |
| 3 | Neha | 35.0 | 60000.0 |

[90]
```
df
```

| | Name | Age | Salary |
|---|---|---|---|
| 0 | Amit | 25.0 | 30000.0 |
| 1 | Rahul | NaN | 45000.0 |
| 2 | Priya | 28.0 | NaN |
| 3 | Neha | 35.0 | 60000.0 |

Next steps: Generate code with df | New interactive sheet

[91]
```
df["Salary"].mean()
```

```
np.float64(45000.0)
```

[93]
```
# Replace the missing values
#df["Salary"] = df["Salary"].fillna(45000)
df["Salary"] = df["Salary"].fillna(df["Salary"].mean())
```

[94]
```
df
```

| | Name | Age | Salary |
|---|---|---|---|
| 0 | Amit | 25.0 | 30000.0 |
| 1 | Rahul | NaN | 45000.0 |
| 2 | Priya | 28.0 | 45000.0 |
| 3 | Neha | 35.0 | 60000.0 |

Next steps: Generate code with df | New interactive sheet

[95]
```
df.duplicated().sum()
```

```
np.int64(0)
```

[97]
```
df.drop_duplicates()   # very importante
```

| | Name | Age | Salary |
|---|---|---|---|
| 0 | Amit | 25.0 | 30000.0 |
| 1 | Rahul | NaN | 45000.0 |
| 2 | Priya | 28.0 | 45000.0 |
| 3 | Neha | 35.0 | 60000.0 |

[98]
```
df["Name"].unique()   # very important
```

```
array(['Amit', 'Rahul', 'Priya', 'Neha'], dtype=object)
```

[99]
```
df["Salary"].unique()
```

```
array([30000., 45000., 60000.])
```

[100]

```
# very important
```

[102]
```
df["Name"].value_counts()   # pivot table  - male ? female ?
```

| Name | count |
|---|---|
| Amit | 1 |
| Rahul | 1 |
| Priya | 1 |
| Neha | 1 |

dtype: int64

[103]
```
df["Salary"].value_counts()
```

| Salary | count |
|---|---|
| 45000.0 | 2 |
| 30000.0 | 1 |
| 60000.0 | 1 |

dtype: int64

[104]
```
df["Salary"].value_counts(normalize=True)*100
# percenatage
```

| Salary | proportion |
|---|---|
| 45000.0 | 50.0 |
| 30000.0 | 25.0 |
| 60000.0 | 25.0 |

dtype: float64

[105]
```
df
```

| | Name | Age | Salary |
|---|---|---|---|
| 0 | Amit | 25.0 | 30000.0 |
| 1 | Rahul | NaN | 45000.0 |
| 2 | Priya | 28.0 | 45000.0 |
| 3 | Neha | 35.0 | 60000.0 |

Next steps: Generate code with df | New interactive sheet

[107]
```
df5 = pd.DataFrame({
    "Name": ["Priya", "Raju", "Sumit"],
    "Gender" :["F", "M", "M"]
})
```

[108]
```
df5
```

| | Name | Gender |
|---|---|---|
| 0 | Priya | F |
| 1 | Raju | M |
| 2 | Sumit | M |

Next steps: Generate code with df5 | New interactive sheet

[112]
```
# Map very important - np.where - we can use it also
```

[113]
```
df5["Gender"] = df5["Gender"].map(
    {
        "M":"Male",
        "F":"Female"
    }
)
```

[114]
```
df5
```

| | Name | Gender |
|---|---|---|
| 0 | Priya | NaN |
| 1 | Raju | NaN |
| 2 | Sumit | NaN |

[115]
```
# .cut - numerical   # very important
# fare category - age category
```

[116]
```
df
```

| | Name | Age | Salary |
|---|---|---|---|
| 0 | Amit | 25.0 | 30000.0 |
| 1 | Rahul | NaN | 45000.0 |
| 2 | Priya | 28.0 | 45000.0 |
| 3 | Neha | 35.0 | 60000.0 |

Next steps: Generate code with df | New interactive sheet

[117]
```
df["Salary_Cat"] = pd.cut(
    df["Salary"] , bins = [0,40000,50000, 60000],
    labels=["Low", "Medum", "High"]
)
```

[118]
```
df
```

| | Name | Age | Salary | Salary_Cat |
|---|---|---|---|---|
| 0 | Amit | 25.0 | 30000.0 | Low |
| 1 | Rahul | NaN | 45000.0 | Medum |
| 2 | Priya | 28.0 | 45000.0 | Medum |
| 3 | Neha | 35.0 | 60000.0 | High |

Next steps: Generate code with df | New interactive sheet

[119]
```
# qcut - quarterly cut -
```

[121]
```
df["SlaryQut"] = pd.qcut(df["Salary"] , q=4,
                         labels =["q1", "q2", "q3", "q4"])
```

[122]
```
df
```

| | Name | Age | Salary | Salary_Cat | SlaryQut |
|---|---|---|---|---|---|
| 0 | Amit | 25.0 | 30000.0 | Low | q1 |
| 1 | Rahul | NaN | 45000.0 | Medum | q2 |
| 2 | Priya | 28.0 | 45000.0 | Medum | q2 |
| 3 | Neha | 35.0 | 60000.0 | High | q4 |

Next steps: Generate code with df | New interactive sheet

[123]
```
# read function
```

[124]
```
df =  pd.read_csv("/content/sample_data/california_housing_train.csv")
```

[125]
```
df
```

| | longitude | latitude | housing_median_age | total_rooms | total_bedrooms | population | households | median_income | median_house_value |
|---|---|---|---|---|---|---|---|---|---|
| 0 | -114.31 | 34.19 | 15.0 | 5612.0 | 1283.0 | 1015.0 | 472.0 | 1.4936 | 66900.0 |
| 1 | -114.47 | 34.40 | 19.0 | 7650.0 | 1901.0 | 1129.0 | 463.0 | 1.8200 | 80100.0 |
| 2 | -114.56 | 33.69 | 17.0 | 720.0 | 174.0 | 333.0 | 117.0 | 1.6509 | 85700.0 |
| 3 | -114.57 | 33.64 | 14.0 | 1501.0 | 337.0 | 515.0 | 226.0 | 3.1917 | 73400.0 |
| 4 | -114.57 | 33.57 | 20.0 | 1454.0 | 326.0 | 624.0 | 262.0 | 1.9250 | 65500.0 |
| ... | ... | ... | ... | ... | ... | ... | ... | ... | ... |
| 16995 | -124.26 | 40.58 | 52.0 | 2217.0 | 394.0 | 907.0 | 369.0 | 2.3571 | 111400.0 |
| 16996 | -124.27 | 40.69 | 36.0 | 2349.0 | 528.0 | 1194.0 | 465.0 | 2.5179 | 79000.0 |
| 16997 | -124.30 | 41.84 | 17.0 | 2677.0 | 531.0 | 1244.0 | 456.0 | 3.0313 | 103600.0 |
| 16998 | -124.30 | 41.80 | 19.0 | 2672.0 | 552.0 | 1298.0 | 478.0 | 1.9797 | 85800.0 |
| 16999 | -124.35 | 40.54 | 52.0 | 1820.0 | 300.0 | 806.0 | 270.0 | 3.0147 | 94600.0 |

17000 rows × 9 columns

Next steps: Generate code with df | New interactive sheet

[126]
```
df1 = pd.read_csv("/content/sample_data/california_housing_train.csv")
```

[127]
```
df1
```

| | longitude | latitude | housing_median_age | total_rooms | total_bedrooms | population | households | median_income | median_house_value |
|---|---|---|---|---|---|---|---|---|---|
| 0 | -114.31 | 34.19 | 15.0 | 5612.0 | 1283.0 | 1015.0 | 472.0 | 1.4936 | 66900.0 |
| 1 | -114.47 | 34.40 | 19.0 | 7650.0 | 1901.0 | 1129.0 | 463.0 | 1.8200 | 80100.0 |
| 2 | -114.56 | 33.69 | 17.0 | 720.0 | 174.0 | 333.0 | 117.0 | 1.6509 | 85700.0 |
| 3 | -114.57 | 33.64 | 14.0 | 1501.0 | 337.0 | 515.0 | 226.0 | 3.1917 | 73400.0 |
| 4 | -114.57 | 33.57 | 20.0 | 1454.0 | 326.0 | 624.0 | 262.0 | 1.9250 | 65500.0 |
| ... | ... | ... | ... | ... | ... | ... | ... | ... | ... |
| 16995 | -124.26 | 40.58 | 52.0 | 2217.0 | 394.0 | 907.0 | 369.0 | 2.3571 | 111400.0 |

| | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| **16996** | -124.27 | 40.69 | 36.0 | 2349.0 | 528.0 | 1194.0 | 465.0 | 2.5179 | 79000.0 |
| **16997** | -124.30 | 41.84 | 17.0 | 2677.0 | 531.0 | 1244.0 | 456.0 | 3.0313 | 103600.0 |
| **16998** | -124.30 | 41.80 | 19.0 | 2672.0 | 552.0 | 1298.0 | 478.0 | 1.9797 | 85800.0 |
| **16999** | -124.35 | 40.54 | 52.0 | 1820.0 | 300.0 | 806.0 | 270.0 | 3.0147 | 94600.0 |

17000 rows × 9 columns

Next steps: Generate code with df1 | New interactive sheet

[128] ✓ 0s

```
df1.to_csv("housingdata.csv")
```

[ ]

Start coding or generate with AI.

Colab paid products - Cancel contracts here

Variables | Terminal | 11:15 AM | Python 3